

Inert Programming

Parameterizing Access to State

Jonathan R. Warden^a

^a Social Protocols

Abstract Context. Most programming languages let any function reach ambient state—globals, built-ins, and system calls. Hidden access undermines local reasoning, testing, portability, and security. Functional programming, dependency injection, and object-capability security each discipline some aspect of state, but they are often treated as separate traditions.

Inquiry. What semantic property of functions and languages makes inversion of control and explicit authority flow the same thing? Prior work already named the function-level restriction: stoicity (Liu, Stucki, Amin, Giarrusso, and Odersky). This essay does not claim that property as new. The remaining question is inertness as a value-level distinction orthogonal to purity, and what follows if the program’s main entry point is stoic.

Approach. This essay is an expository synthesis. It keeps a simple operational vocabulary—access versus interaction, live versus inert—and lifts the function-level rule to a language-level design point: an inert ambient scope and a stoic main. It applies that distinction to both imperative and purely functional settings, including effect values that are pure yet live.

Knowledge. Purity restricts interaction with observable state; hermeticity, which is stoicity at the function level, restricts access. A function can be pure and still hard-wired to the real console, clock, or filesystem. A hermetic function can be impure if it interacts only through parameters. Live values provide access to state; inert values do not. Mutability is not the axis. Making main stoic forces packages and the ambient scope to be inert, so authority enters only through explicit interfaces. Capture checking is more flexible than this syntactic discipline; it is not claimed moot, and it is not the topic.

Grounding. The argument is definitional and comparative. It is illustrated with small programs in TypeScript, Go, Python, and Haskell, and situated against stoicity, Gordon’s use/mention distinction, Joe-E, Melicher and colleagues’ pure modules, and common library patterns such as Go’s net/http, Rust’s cap-std, and sans-I/O. An appendix characterizes liveness with a mockability test that does not assume an object-capability model.

Importance. Treating inertness as orthogonal to purity gives a shared vocabulary across paradigms and explains why capability discipline still matters in languages such as Haskell. A stoic main is a simple whole-program baseline: no ambient authority, function signatures as dependency manifests, and benefits for reasoning, testing, portability, and security.

ACM CCS 2012

- **Software and its engineering** → **Language features**; *General programming languages*;
- **Security and privacy** → *Software security engineering*;

Keywords inertness, stoicity, hermeticity, capability security, purity

The Art, Science, and Engineering of Programming

Perspective The Art of Programming

Area of Submission Programming languages, Language design



© Jonathan R. Warden
This work is licensed under a “CC BY 4.0” license
Submitted to *The Art, Science, and Engineering of Programming*.

1 Introduction

In most programming languages, any function can reach out and touch the world: read the clock, write a file, open a socket. But ambient access to state—through singletons, globals, built-in functions, and system calls—makes testing brittle and reasoning murky. You run a test once and it passes; run it again and it fails because the clock advanced or a temporary file wasn't deleted. You call out to a small math library and it exfiltrates your SSH keys because it had access to your home directory.

Several programming disciplines help tame access to state. Functional programming forbids interaction with state during evaluation, while dependency injection [Fowler 2004] allows interaction but makes dependencies on stateful resources explicit.

What if a language took dependency injection to its logical conclusion, making it a **semantic property** of code—like purity—rather than just a design pattern? Then all system resources would have to be injected—for example, as parameters passed to `main`.

Example (TypeScript)

```
1 import type { Console } from "io";
2
3 // Runtime injects the concrete Console implementation.
4 export function main(console: Console): number {
5   console.println("Hello, World!");
6   return 0;
7 }
```

Here, `main` is a **hermetic function**:

A function is hermetic iff it does not access existing state except through its parameters.

This function-level restriction is stoicity [Liu et al. 2020]. This essay does not claim that property as new. What remains is inertness as a property of values, and the consequences of a stoic `main`.

If `main` is hermetic, then any function it depends on must also be hermetic—otherwise `main` is indirectly accessing existing state. So a hermetic `main` eliminates **ambient authority** [Yee et al. 2003] by requiring all capabilities to be explicitly passed as parameters. This aligns with the defining discipline of **object-capability languages** [Miller 2006], where authority over state is conveyed by explicit, unforgeable references. If both “dependencies” and “authority” are taken to include *any stateful resource*, then “inject all dependencies” and “explicit capability passing” become the same language property. That property is obtained by making `main` hermetic.

In a **hermetic programming language**, function parameters act like hermetically sealed channels through which all access to state flows. Whether writing to a file, reading a channel, or mutating a buffer, the caller controls the world the function can see. Deterministic time? Pass a fake clock. Capture standard output? Pass a mock console. Every potential access to state is visible at the call boundary. Function signatures become dependency manifests. No hidden inputs. No undeclared effects.

2 The Purity Gap

Hermeticity is a semantic property of functions or languages, similar to purity. But hermeticity is not just purity-lite. Purity restricts *interaction* with state, while hermeticity restricts *access*. These are orthogonal.

Suppose the function `getClock(): Clock` returns a global opaque handle to the system clock. `getClock` is pure: it has no effect on the clock, and because it always returns the same constant, it is referentially transparent.

Now suppose the function `getTime(clock: Clock): Time` takes a `Clock`—either the real clock or a mock—and returns that clock’s time. The current time can then be obtained from the system clock by calling `getTime(getClock())`.

`getTime` is hermetic because it accesses no state other than the clock passed as a parameter. `getClock` is pure. But the composed function `now = getTime ∘ getClock` is *neither hermetic nor pure*.

How is that possible?

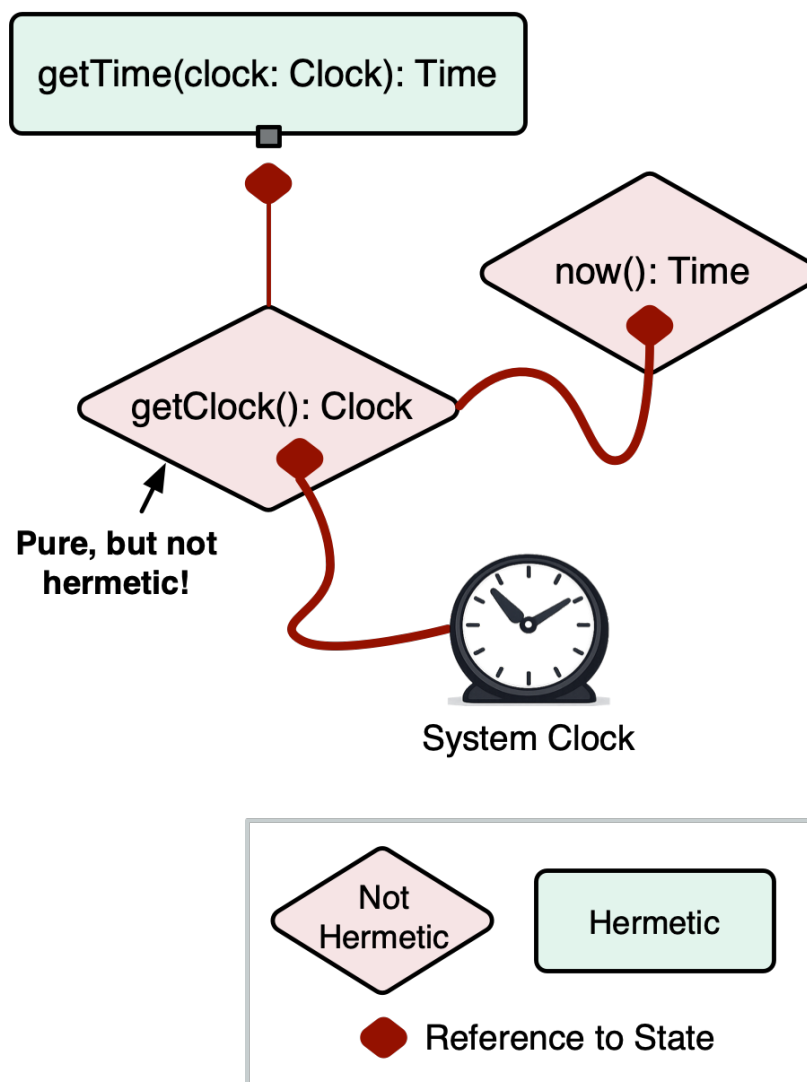
Because `getClock`, while pure, is still tainted by **access** to ambient state. By returning a reference to the real clock, it **exposes** that state.

3 Interaction vs Access

`now` and `getClock` are both **hard-wired** to state: they access the system clock, and that access cannot be redirected by passing a different parameter.

`getTime`, by contrast, parameterizes its access to state: it is hermetic.

Inert Programming



■ **Figure 1** Illustration of hermetic vs non-hermetic functions. Non-hermetic functions are hard-wired to state.

It is *interaction* with state that makes a function impure, not access. The most widely accepted definition of purity is **referential transparency**: an expression can be replaced by its value in any program context without changing observable[†] behavior. A function fails referential transparency when evaluation interacts with **observable state**: either it *affects* state, or it is *affected by* state and so can return different results for the same inputs.

So hermeticity is both more and less strict than purity. A hermetic function may interact with the world as long as it is not hard-wired to it. A pure function may be hard-wired to the world as long as it does not interact with it.

[†] As usual, we ignore changes not observable by normal program operations, such as internal allocation, garbage collection, caching, or timing effects. Allocating fresh state is internal to the call unless it escapes, assuming ordinary memory safety.

Gordon’s use/mention distinction is useful here: purity is a use/interaction property, while inertness is a mention/access property [Gordon 2020]. A live value can still be referentially transparent. Mutability is not the axis—an immutable or read-only value may still convey observational authority, and in a pure functional setting mutability is the wrong category. Scala’s capture-checking documentation uses “pure” for types with empty capture sets; that is internally coherent, but it overloads a term that traditionally means something else in functional programming.

Inert Programming

Interaction vs Access Grid: Examples

		access	
		hermetic	non-hermetic
interaction	pure	<code>sqrt(x)</code>	<code>getClock()</code>
	impure	<code>getTime(clock)</code>	<code>now()</code>

■ **Figure 2** Example functions arranged along the axes of interaction (pure/impure) and access (hermetic/non-hermetic).

This orthogonality also applies at the programming-language level. This means even a pure functional programming language is not necessarily hermetic (and most FP languages are not hermetic). To understand the implications of hermeticity for the design of programming languages and libraries, and the benefits it offers independently of purity, we first need a more precise vocabulary for how functions expose access to state.

Terms used so far in this essay:

- **state**: anything that can observably affect or be affected by a computation
- **existing state**: state that existed before the function call
- **observable state**: state observable outside the call, including fresh state that escapes
- **parameterized state**: existing state accessed through a function's parameters
- **interact**: to **affect or be affected by** state
- **expose**: to return a live value or write it into **observable state**
- **access**: to **interact with or expose** state
- **pure**: no **interaction** with **observable state**
- **hermetic**: no **access** to **non-parameterized state** (stoicity; not claimed as new)

4 Live and Inert Values

`getClock` exposes state by returning a handle. A function could also expose the same handle by writing it to a channel or storing it in a mutable data structure. In any case, what is exposed is a value that provides access to state.

*A value that provides access to state is **live**.*

Live values can include object references, handles, primitives, closures, and anything that embeds any of these and thereby provides a path to state.

Values that do not provide access to state are **inert**.

4.1 Providing Access to State

Providing access to state is different from merely **designating** state. A filename, for example, does not by itself provide the ability to interact with the file system. A function that receives a filename would still need to reach out to some library or builtin such as `open`, in which case `open` is the live value.

Intuitively, a value is live if the interactions it enables can be redirected by swapping it for a mock backed by caller-controlled in-memory state. In Appendix D, I formalize this as the **Mockability Test**.

In languages with well-defined notions of objects and references, the live/inert distinction can often be characterized in terms of object-graph reachability. For example, Joe-E’s **immutable** objects are inert by construction: they are immutable, and no mutable object or external state is reachable from them.²

4.2 Functions as Values

We can talk about functions in two roles: as code (callable), and as values (passable or storable).

A function is hermetic as code exactly when it is inert as a value.

An inert function does not provide access to state, so passing it to another function cannot provide that function with access to state. Conversely, a non-hermetic function is live.

A function is live iff its definition embeds a live value; specifically, it must contain a **live free identifier**—a free identifier referencing a live value—where a **free identifier** is any name appearing in a function definition that is not a parameter or local variable.

Consider the two Python functions below:

² Joe-E defines immutable objects transitively: their fields, and objects reachable from them, must not change after construction. Such objects are inert in the sense used here, though *inert* is broader than Joe-E’s object-graph formulation and is not limited to object-oriented languages [Mettler et al. 2009].

Inert Programming

```
from typing import TextIO

def hello(out: TextIO) -> None:
    out.write("Hello, World!\n")
    // out is bound to a parameter. So hello is inert.

from sys import stdout

def main() -> None:
    hello(stdout)
    // stdout is free, so main is live.
```

<u>live</u> free <u>live + free</u>
--

■ **Figure 3** Example of inert vs live function definitions.

The only lexical name that `hello` references is the parameter `out` (`write` is a member selection on `out`), so `hello` is inert. In contrast, `main` is live because it refers to the live free identifier `stdout`.

5 Hermetic Programming Languages

There are two sources of live free identifiers in a language:

- **ambient identifiers**: globals, imports, builtins, primitives
- **captured environments**: closures

5.1 Inert Ambient Scope

Ambient identifiers are names available to all functions and modules by default: preludes, built-in functions, default imports, global constants, system calls, and so on. Collectively, these form the **ambient scope**.

*A hermetic programming language implies an **inert ambient scope**.*

Dependency injection, object-capability discipline, Joe-E's universal scope, and Melicher et al.'s pure modules already describe this idea [Fowler 2004; Miller 2006; Mettler et al. 2009; Melicher et al. 2017]. The design question this essay takes up is what follows if `main` is stoic.

If the ambient scope contains even one live identifier, then any function can reach out and access ambient state.

5.1.1 Inert Packages

This means that imports must not introduce live values into ambient scope, nor have observable side effects during initialization. In that sense, *all packages must be inert*.

Package-scoped functions and values therefore cannot capture live values from other packages.

Example (Go): a live package capturing a live value from another package

```

1 package logger
2
3 import "os"
4
5 func Log(msg string) {
6     os.Stdout.WriteString(msg + "\n")
7 }
```

Here, `Log` is live because it captures the live free identifier `os.Stdout`.

Packages also cannot host global singletons; package-level globals and exported values must be inert.

Inert packages can, however, export hermetic constructors that **mint** fresh state without accessing existing state. They may also export inert types, interfaces, methods, and other definitions for complex data structures and algorithms. They may even provide logic for interacting with external resources such as databases or web services, as long as access to those resources is passed as parameters.

In a hermetic programming language, the standard library defines interfaces to system resources such as the filesystem, network, and clock, but actual access happens only through injected parameters. Many existing libraries implement this pattern: Go's `http.Serve` accesses the network through its `net.Listener` parameter, Rust's `cap_std` routes I/O through capability values, and Python sans-I/O libraries such as `hyper-h2` go further by factoring I/O out entirely [Go net/http docs; `cap-std`; Benfield 2017].

5.2 Closures

An inert ambient scope prevents functions from capturing live *ambient identifiers*. But it does not prevent closures from capturing live *local identifiers*. Making all functions hermetic therefore additionally requires eliminating **live closures**.

So languages face a design choice:

1. **Allow live closures:** higher-order function values may carry captured authority. This may be desirable in languages where partial application and higher-order functions are idiomatic.
2. **Take the closures-as-objects view:** a closure with a hidden environment is treated not as a function value, but as an object with a hermetic `apply` method.³

Capture checking is more flexible than this syntactic discipline; this essay does not claim it is moot, and it is not the topic here.

³ The closures-as-objects view is the language-design analog of *defunctionalization* [Reynolds 1972/1998].

Inert Programming

5.3 Hermetic Language Properties

The defining requirement of a hermetic programming language is a hermetic main function, whether or not it is literally called `main`.

An inert ambient scope forces `main` to be hermetic. Conversely, if `main` is hermetic, then any live identifiers in ambient scope are necessarily unused.

Hermetic Programming Language = Hermetic Main Function $\approx$ Inert Ambient Scope

This still leaves one possible source of non-hermetic function values: captured local environments.

All Functions Hermetic = Hermetic Programming Language + No Live Closures

6 Hermetic Programming Benefits

6.1 Behavioral Referential Transparency

For pure functions, referential transparency means that $f(x)$ depends only on the **value** of x (and the definition of f). **Behavioral referential transparency** extends this idea to stateful inputs: $f(x)$ depends only on the observable **behavior** of x . There are no hidden inputs.

This facilitates **local reasoning**: minimizing the things a programmer must keep in mind to understand a fragment of code.

Pure functional programming achieves a particularly strong form of local reasoning by eliminating interaction with state entirely. But hermeticity also improves local reasoning by reducing the splash radius of possible interaction to the state reachable through explicit function parameters.

For example, suppose I pass a mutable list to a hermetic function:

```
1 // x is a mutable list of numbers
2 let x: number[] = [1, 2, 3]
3 f(x)
```

If f is hermetic, the only state it can access is the list referenced by x . It cannot consult a global, log to a singleton, or touch the clock. To understand $f(x)$, the only state I need to think about is the content of that list.

6.2 Composability

Purity composes: if f and g are pure, then $f \circ g$ is pure.

Hermeticity composes too. Hermetic functions cannot grant one another access to existing state. So if f and g are hermetic, then $f \circ g$ is hermetic.

This means large programs can be assembled from small pieces that remain hermetic all the way up to `main`.

6.3 Testability, Determinism, and Portability

Hermeticity improves **testability** because stateful dependencies are already explicit and can be replaced with mocks without additional refactoring. It improves **determinism** because sources of variation—clock, RNG, network, filesystem, environment—must be passed explicitly and can therefore be replaced with deterministic alternatives. And it improves **portability** because hermetic code has no hard-coded environmental dependencies: it can be reused across execution contexts such as plugin systems, browsers, or smart contracts without depending on a special sandbox merely to prevent ambient access.

6.4 Security

Finally, hermetic programming changes the application’s trust model. By forcing authority to enter only through explicit interfaces, it naturally aligns with the core discipline of **capability-based security**, which I explore next.

7 Capability-Based Security

Today, we routinely download thousands of transitive dependencies through package managers like npm or cargo, trusting that none have been compromised. Because most languages grant ambient authority to the network and filesystem by default, an attacker who hijacks a package can silently exfiltrate environment variables, SSH keys, or database credentials.

But why should a random math library have access to your filesystem?

One of the core ideas of capability-based security is the elimination of **ambient authority**: those pernicious “pools of authority on which viruses grow” [Miller and Shapiro 2003]. In a language without ambient authority, untrusted code simply *cannot* access the network or disk unless it was explicitly given the capabilities required to do so. This also helps limit the surface of arbitrary-code-execution attacks.

Hermetic programming languages enforce the “no ambient authority” rule as a semantic language property.

7.1 Live Values Are Capabilities

Since a hermetic function by definition may access existing state only through its parameters, access to existing state cannot be obtained by forging references. Receiving a live parameter must be the only way a function can obtain authority over existing state.

In a hermetic programming language, live values are capabilities.

Inert Programming

7.2 Delegation, Revocation, and Confinement

“Ambient authority” is sometimes taken to mean only ambient access to system resources. But if a function can communicate through a pre-existing channel, or write a live value into existing mutable state for later retrieval, then authority can flow between calls without appearing in the signature.

A hermetic programming language rules that out by eliminating ambient authority to *any existing state*. A hermetic function does not have its own memory; it can only delegate authority through channels explicitly provided by the caller.⁴ If a function is never given the authority to write a capability into existing state—a way of exposing state that I call **grafting**—then the authority it receives remains **overtly confined**⁵ to that unit of work and is automatically **revoked** [Yee et al. 2003] once the function finishes its work.

These are not extra security mechanisms: they follow from the core semantics of hermetic functions.

7.3 Principle of Least Authority

Eliminating *ambient authority* does not prevent programmers from granting *too much authority*. If `main(world)` is injected with an object containing `world.clock`, `world.network`, and so on, it can still pass that “god object” down through the call stack. This may be hermetic, but it is poor security hygiene.

The **principle of least authority** (POLA) [Miller 2006] dictates that `main` should ask for—and the host should grant—only the capabilities the program actually needs. In a hermetic language, the signature of the program’s main function acts as a dependency manifest, and capability-oriented interface standards such as WASI [WASI] can be used as a portable vocabulary for expressing those dependencies.

POLA also requires **attenuating** capabilities before passing them onward [Miller 2006]: pass a single file handle instead of the whole filesystem, and make it read-only if possible.

The same principle applies to meta-authority: authority to delegate or persist authority. Even in a hermetic language, a function can communicate or remember a capability by grafting a live value into mutable state broad enough to hold live values.

⁴ Miller, Yee, and Shapiro identify **Property F: Access-Controlled Delegation Channels**: authority flows only along channels that are themselves access-controlled [Yee et al. 2003].

⁵ The classic **confinement problem** is ensuring that a program cannot transmit information except through authorized channels [Lampson 1973].

Example (Go): authority delegation by grafting

```

1 func handle(ctx map[string]any, db *Database) {
2     ctx["db"] = db // grafted: db is now reachable via ctx
3 }

```

In this example, any code that reads `ctx` now has database access.

A more secure design would attenuate the capabilities provided not just by `db` but also by `ctx`, wrapping `ctx` to make it read-only or restricting the types it can store.

7.4 Hermeticity and Ambient Authority

Although a hermetic programming language has no ambient authority, the converse is not true: *a language can have no ambient authority without being hermetic*. Hermeticity corresponds to a stronger capability discipline: authority must flow explicitly through capabilities passed as references or parameters.

A capability-secure language can also introduce authority through **ambient endowment**. For example, SES compartments have no ambient authority by default, but the host can endow them with live globals or modules [Endo SES]. WASI likewise provisions capabilities through imports. In both cases, authority is supplied by the host through ambient names rather than through function parameters.

By contrast, in Joe-E, explicitly propagated references are the only things that convey authority; *the universal scope provides no authority* [Mettler et al. 2009; Wagner 2007]. This maps closely to what I call an **inert ambient scope**. In that sense, Joe-E can be classified as a hermetic programming language.

8 Hermetic Programming in Pure Functional Languages

You might think hermeticity is irrelevant in a pure language. If functions cannot have side effects, why worry about access to state?

Because pure values can still be **live**.

8.1 Effect Values Can Be Live

In Haskell, an `IO ()` describes an effectful computation [Peyton Jones 2001]. Constructing it has no side effects, but when interpreted by the runtime it interacts with state.

```

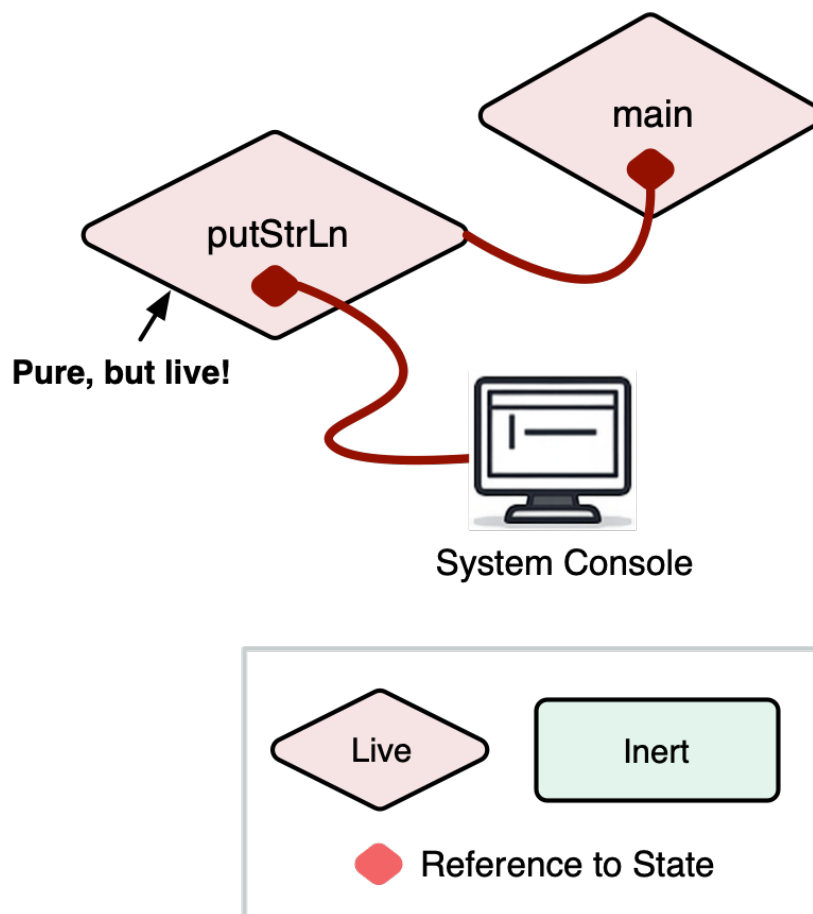
1 main :: IO ()
2 main = putStrLn "Hello, World!"

```

Here, `main` is a pure value. But it is live in this essay's sense because it is already committed to the real console instead of receiving console access as a parameter. It already embeds authority to interact with existing state.

Pure does not imply inert.

Inert Programming



■ **Figure 4** Illustration of a Hello, World! program where **main** is live because it is ultimately hard-wired to the system console.

putStrLn is also live by the Mockability Test: it could be replaced by a function with the same type that redirects console writes to an in-memory buffer, without otherwise changing observable program behavior.

8.2 The Hermetic Alternative

Compare **main** with a version that parameterizes its access to the console using the **tagless final** pattern [Carette et al. 2009]:

```
1 class Monad m => Console m where
2   putLine :: String -> m ()
3
4   hermeticMain :: Console m => m ()
5   hermeticMain = putLine "Hello, World!"
```

hermeticMain does not assume a particular console and does not commit to IO. The caller can supply either an in-memory mock or the real console through a live **Console** IO instance:

```

1 instance Console IO where
2   putLine = putStrLn
3
4 main :: IO ()
5 main = hermeticMain

```

So `hermeticMain` is both pure and inert, whereas `main :: IO ()` is pure but live. Both sit in the “pure” row of the interaction-vs-access grid—but in different columns.

8.3 Toward Hermetic Haskell

Haskell’s Prelude and standard library contain many live identifiers: `putStrLn`, `readFile`, `getCurrentTime`. Any function that uses one of these becomes live. Liveness then propagates through composition, just as it does in imperative languages.

A “Hermetic Haskell” would move such live identifiers out of the Prelude and standard libraries, exporting only interfaces to system resources, with concrete implementations injected into `main` by the runtime. The `ReaderT` pattern can help manage the extra wiring [Snoyman 2017].

Purity guarantees that code does not interact with observable state when evaluated. Hermeticity guarantees that code does not have *access* to observable state unless authorized. A function can be pure but still hard-wired to the real filesystem, clock, or network. That is why patterns like tagless `final` and `ReaderT` are common in Haskell: not because Haskell is not pure enough, but because purity and hermeticity solve different problems.

9 Conclusion

When we think of “pure” data, we may imagine something cleanly serializable into a format like JSON. But the quality we are really reaching for is not purity but **inertness**. References, channels, closures, and effect values are the living machinery of computation, rooted in their execution environment. Integers, strings, and lists are inert matter being computed.

Hermeticity is inertness applied to functions. The function-level rule is stoicity; this essay does not claim that rule as new. A hermetic function may not be pure, but it is pure functionality. Since it cannot access state directly, it must be plugged into live parameters to do anything in the world.

The inert/live distinction applies even in pure functional languages. Inert/live and pure/impure are independent axes of “clean.” Hermetic and functional programming are complementary.

Making a programming language hermetic is straightforward in principle: keep the ambient scope inert, export interfaces instead of live values from standard libraries, and inject concrete resources into the main function. Contexts or implicits can manage the extra wiring.

Inert Programming

Parameterizing access to state carves programs at the joints. Hermetic functions are decoupled from free state, testable against mocks, portable, and composable. Function signatures are dependency manifests. All authority is explicit.

We should look back at global singletons or `import fs` granting immediate, permissions-based disk access with the same horror we feel for `goto`. The next generation of programming languages should be hermetic.

Hermetic programming links inversion of control, capability-based security, and local reasoning through a single semantic rule: **a function may access existing state only through its parameters**. No hard-coded dependencies. No ambient authority. No hidden inputs. No leaks.

10 Appendices

10.1 Appendix A: Hermeticity as Isolation

10.1.1 Etymology of “Hermetic”

The term **hermetic** has been used to describe a number of systems that isolate imperative code from its environment: most notably Google’s **hermetic testing**, **hermetic servers**, hermetic builds such as Bazel, and hermetic languages such as Wuffs and Starlark [Herbert 2010; Google Testing Blog 2012; Bazel; Wuffs; Starlark]. These all **isolate** effects to specifically authorized state: a test mock, a build artifact, the function’s arguments.

I have appropriated the term *hermetic* in this essay to mean isolation only with respect to *access* to state. But there are levels of isolation beyond hermeticity. For example, Wuffs programs are also isolated in the ACID sense, and they cannot spawn threads, allocate memory, or panic.

10.1.2 Call-Boundedness

A hermetic function might still spawn a goroutine or task that keeps interacting with state after the function has completed. It is still hermetic as long as the task interacts only with explicitly authorized state.

A function that uses **structured concurrency** ensures that all spawned threads complete before the function returns [Smith 2018]. I call such a function **call-bound**. Just as a pure function can use **internal state** that does not survive the call, a call-bound function may use **internal concurrency** that does not survive the call.

With call-bound hermetic functions, the function call is a boundary in both space and time: authority is granted to specific resources for the duration of the call only.

10.1.3 Atomicity

In a multithreaded environment with shared state, hermeticity does not guarantee **isolation in the ACID sense**: preventing concurrent processes from interleaving reads and writes.

A function call is **atomic** if, from the caller’s perspective, any state changes happen in a single instant—or not at all.

10.1.4 Containment

Hermeticity does not guarantee that a function will **complete successfully**: it may loop forever, run out of memory, panic, and so on.

A function is **contained** if it is guaranteed to return control to the caller without crashing the host process, regardless of input. This can be achieved through a language that prevents runtime panics, as Wuffs does, or through a runtime that sandboxes execution, as Erlang supervisors do.

10.1.5 Full Isolation

Hermeticity and call-boundedness are properties of the function definition itself, while atomicity and containment depend on the execution context. A function that is

Inert Programming

hermetic and call-bound, run in a context that guarantees atomicity and containment, could be considered **fully isolated**.

10.2 Appendix B: Restrictions on Access to State

The definitions of pure and hermetic functions lead to some initially surprising conclusions. For example, constructors that return live values can still be hermetic.

This appendix summarizes the different types of state and the restrictions on how pure and hermetic functions can access them.

10.2.1 Types of State

This essay defines **state** as anything that can affect or be affected by a computation in a way that is observable by normal program operations, ignoring timing, memory, and CPU usage. This is the same notion of observability typically used in definitions of purity and referential transparency. Equivalently, state is anything that, if interacted with, can make an expression non-referentially transparent.

State can exist before a function call (**existing state**) or be allocated during the call (**fresh state**).

Fresh state that does not escape the dynamic extent of the call is **internal state**.

Existing state, and fresh state that does escape the call, are **observable state**: their contents or identity may still matter after the function returns.

10.2.2 Grafting State

A function can expose state by writing a live value into existing state—for example, storing a reference in a mutable field or writing it to a channel—thereby making it reachable by other code without returning it. I call this **grafting** state.

10.2.3 Minting State

A hermetic function can expose **fresh** state that it allocates during the call. I call this **minting** state. Minting is why constructors can be hermetic: the minted state did not exist before the call, so the function is not accessing existing state. The function as a value remains inert.

Summary of Restrictions

Capability	Pure	Hermetic
Access free state	Yes	No
Interact with observable state	No	Yes
Interact with internal state	Yes	Yes
Graft state	No	Yes
Mint state	No	Yes

10.3 Appendix C: Glossary of Terms

- **state**: anything that can affect or be affected by a computation in a way that is observable by normal program operations
- To **allocate** state: to allocate memory that can be read and written
- **existing state**: state that existed before the function call
- **fresh state**: state allocated during the function call
- **internal state**: fresh state that does not escape the function call
- **observable state**: state, fresh or existing, that is observable outside the call
- **parameterized state**: state accessed through a live value received as a parameter
- **free state**: existing state that is not parameterized
- **ambient state**: free state accessible from ambient scope
- **live**: provides access to state, per the Mockability Test
- **inert**: not live
- **interact**: to **affect or be affected by** state
- **access**: to **interact with or expose** state
- **expose**: to provide access to state by returning or grafting a live value
- **graft**: to write a live value into observable state
- **mint**: to expose fresh state by causing it to escape
- **pure**: no **interaction** with **observable state**
- **hermetic**: no **access** to **free state** (stoicity; not claimed as new)
- **identifier**: any binding or name-to-value association that can be referred to in a function definition
- **live identifier**: any identifier whose value is live
- **free identifier**, relative to a function definition: any identifier that is not a parameter or local variable
- **ambient identifier**: a free identifier available to all functions
- **ambient scope**: the collection of all ambient identifiers available to a function

10.4 Appendix D: The Mockability Test

We need a definition of **live value** that works in *any* language, without assuming object-capability discipline or appealing to less formal definitions of authority. The only thing we assume is an operational view of evaluation: running code can produce an **interaction trace**—a log of observable interactions with **existing state**, such as the filesystem, network, or clock, ignoring timing and resource usage.

10.4.1 The Trace-Projection Idea

A program’s interaction trace can be partitioned by *which state it touches*. If S is a particular existing state resource—or set of resources—such as a specific file, socket, clock, or region of memory, write:

- $\pi_S(\text{trace})$ for the **projection** of a trace onto only the events that interact with S .

This lets us express “this program interacts with S ” without talking about reference graphs or authority: it simply means $\pi_S(\text{trace})$ is non-empty.

Inert Programming

10.4.2 Definition of Liveness

Let program contexts $C[-]$ range over **expression contexts**: the hole may appear wherever an expression may appear, including in operator position, as in $C[-] = (-)(x)$. This covers both `read(x)` and method-style calls once desugared: for example, `x.read()` selects some callable expression and then applies it.

A value v provides access to some existing state S iff:

*There exists a substitute value v' , admissible in place of v , such that, for every program context $C[-]$ whose behavior does not depend on the representation identity of the value plugged into the hole—for example, its pointer address—the projected traces $\pi_S(\text{trace}(C[v]))$ and $\pi_{S'}(\text{trace}(C[v']))$ are the same **up to renaming of state identities**, where S' ranges over **fresh caller-allocated internal state** disjoint from S .*

And:

v is live iff it provides access to some existing state S .

Intuitively: v provides access to S if whatever interactions a program can have with S can always be retargeted, by swapping v for v' , to an isomorphic interaction with state that the caller allocates and controls.

10.4.3 Why “Internal State” Matters

The requirement that S' range over the caller’s **internal state** is the true test of whether a value really provides access to state, rather than merely designating it. For example, if the caller of `read(filehandle)` can redirect reads to a different file but cannot redirect them to an in-memory mock, then the *filehandle token* is just a **designator**. The live part is whatever operation interprets that token—such as `read` or `filehandle.read`—because *that* is what must be substituted to retarget the interaction to caller-controlled internal state.

If, on the other hand, `read` accepts an interface, trait, or function that the caller can implement, then the caller can substitute a behavioral equivalent that routes reads not just to a different file, but to any internal state the caller controls. In that case, the file handle value itself is live.

10.4.4 Pointers Are Live

A pointer to mutable state is live under this test because it is retargetable: in any context that treats pointers extensionally, rather than inspecting their numeric addresses, a pointer to one region can be substituted with a pointer to a fresh region, yielding an isomorphic interaction trace over that region.

This remains true even in a memory-unsafe language where pointers are **forgeable** from integers. That is not a contradiction: **liveness** is about whether a value can serve as a retargetable conduit to state. **Hermeticity** is a stronger language property: it requires eliminating *ambient* ways to obtain conduits. In a language like C, the dereference operator (`*`) and integer-to-pointer casts act as ambient authority over memory.

A hermetic programming language must therefore make live values **unforgeable**—that is, it must provide some form of memory safety or capability safety—so that a program cannot conjure access to arbitrary memory out of thin air.

10.5 Appendix E: Closures and Methods

10.5.1 Closures

A closure that captures a live value is itself live and therefore not hermetic. However, a function that *receives* a live closure as a parameter can still be hermetic: the state access is parameterized by the closure value. Similarly, a hermetic function can create and return a new closure, as long as that closure does not capture any live free identifiers relative to the enclosing function's body.

Since all package-scoped functions are inert in a hermetic programming language, the only way a live function can enter a program is as a locally scoped closure or nested function that captures live local variables.

10.5.2 Methods

Methods can be thought of as functions parameterized by their receivers. A method that only interacts with state reachable through its receiver and other parameters is hermetic: the *object* is live, not the method.

Consider an object that embeds a reference to a logger. If one of the object's methods writes to that logger, and only to that logger, the method is hermetic. On the other hand, if the method hard-codes access to a global logger instead of the one embedded in the object, then the method itself is live.

It follows that in a hermetic programming language, exported types must be hermetic in the sense that they cannot have live methods.

11 References

- [**Bazel**] Bazel documentation, *Hermeticity*. <https://bazel.build/basics/hermeticity>
- [**Benfield 2017**] Cory Benfield, *Sans-IO: Network Protocol Libraries in Python* (2017). <https://sans-io.readthedocs.io/>
- [**cap-std**] Bytecode Alliance, cap-std: capability-based standard library for Rust. <https://github.com/bytecodealliance/cap-std>; <https://docs.rs/cap-std>
- [**Carette et al. 2009**] Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan, *Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages* (Journal of Functional Programming, 2009). <https://okmij.org/ftp/tagless-final/JFP.pdf>
- [**Finifter et al. 2008**] Matthew Finifter, Adrian Mettler, Naveen Sastry, and David Wagner, *Verifiable Functional Purity in Java* (CCS 2008), pp. 161–174. <https://people.eecs.berkeley.edu/~daw/papers/pure-ccs08.pdf>
- [**Fowler 2004**] Martin Fowler, *Inversion of Control Containers and the Dependency Injection pattern* (2004). <https://martinfowler.com/articles/injection.html>
- [**Gordon 2020**] Colin S. Gordon, *Designing with Static Capabilities and Effects: Use, Mention, and Invariants* (ECOOP 2020). <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2020.10>
- [**Go net/http docs**] Go standard library documentation, net/http.Serve. <https://pkg.go.dev/net/http#Serve>
- [**Google Testing Blog 2012**] Google Testing Blog, *Hermetic Servers* (2012). <https://testing.googleblog.com/2012/10/hermetic-servers.html>
- [**Herbert 2010**] Scott Herbert, *Benefits of Hermeticity* (2010). <https://slaptijack.com/programming/benefits-of-hermeticity.html>
- [**hyper-h2**] Python Hyper Project, hyper-h2. <https://github.com/python-hyper/h2>
- [**Lampson 1973**] Butler W. Lampson, *A Note on the Confinement Problem* (Communications of the ACM, 1973). <https://www.cs.cornell.edu/andru/cs711/2003fa/reading/lampson73note.pdf>
- [**Liu et al. 2020**] Fengyun Liu, Sandro Stucki, Nada Amin, Paolo G. Giarrusso, and Martin Odersky, *Stoic: Towards Disciplined Capabilities* (2020). <https://infoscience.epfl.ch/handle/20.500.14299/164482>
- [**Melicher et al. 2017**] Darya Melicher, Yangqingwei Shi, Alex Potanin, and Jonathan Aldrich, *A Capability-Based Module System for Authority Control* (ECOOP 2017). <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2017.20>
- [**Mettler et al. 2009**] Adrian Mettler, Tyler Close, and David Wagner, *Joe-E Specification* (2009). <https://people.eecs.berkeley.edu/~daw/joe-e/spec-20090918.pdf>
- [**Miller 2006**] Mark S. Miller, *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control* (2006). <https://www.erights.org/talks/thesis/markm-thesis.pdf>
- [**Miller and Shapiro 2003**] Mark S. Miller and Jonathan S. Shapiro, *Paradigm Regained: Abstraction Mechanisms for Access Control* (ASIAN 2003), pp. 224–242. <http://www.erights.org/talks/asian03/paradigm-revised.pdf>
- [**Peyton Jones 2001**] Simon Peyton Jones, *Tackling the Awkward Squad: monadic input/output, concurrency, exceptions, and foreign-language calls in Haskell* (2001). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/mark.pdf>

- [**Reynolds 1972/1998**] John C. Reynolds, *Definitional Interpreters for Higher-Order Programming Languages* (originally presented 1972; reprinted 1998). <https://link.springer.com/content/pdf/10.1023/A:1010027404223.pdf>
- [**Endo SES**] Endo, *SES (Hardened JavaScript)*. <https://github.com/endojs/endo/blob/master/packages/ses/README.md>
- [**Smith 2018**] Nathaniel J. Smith, *Notes on structured concurrency, or: Go statement considered harmful* (2018). <https://vorp.us.org/blog/notes-on-structured-concurrency-or-go-statement-considered-harmful/>
- [**Snoyman 2017**] Michael Snoyman, *The ReaderT Design Pattern* (2017). <https://www.fpcomplete.com/blog/readert-design-pattern/>
- [**Starlark**] Bazel Build, Starlark. <https://github.com/bazelbuild/starlark>
- [**Wagner 2007**] David Wagner, TRUSTo7 slides on Joe-E. <https://people.eecs.berkeley.edu/~daw/talks/TRUST07.pdf>
- [**WASI**] WebAssembly System Interface, *Capabilities*. <https://wasi.dev>; <https://github.com/WebAssembly/WASI/blob/main/docs/Capabilities.md>
- [**Wuffs**] Google, Wuffs: Wrangling Untrusted File Formats Safely. <https://github.com/google/wuffs>
- [**Yee et al. 2003**] Ka-Ping Yee, Mark S. Miller, and Jonathan Shapiro, *Capability Myths Demolished* (Systems Research Laboratory Technical Report SRL2003-02, Johns Hopkins University, 2003). <https://srl.cs.jhu.edu/pubs/SRL2003-02.pdf>

Inert Programming

About the author

Jonathan R. Warden Contact him at john.warden@gmail.com.